Project 2 - Student Performance Analysis

Prepared by: Rizwan Ali | Data Analyst Internship, Syntecxhub

1. Introduction

This project analyzes the academic performance of 300 students across five subjects (Math, English, Science, Urdu, Computer) using Python (pandas, numpy, matplotlib). It covers data preprocessing, subject-wise and overall trends, factors affecting performance, class comparison, visualization and recommendations. The dataset is a simulated student dataset generated with a fixed random seed for reproducibility.

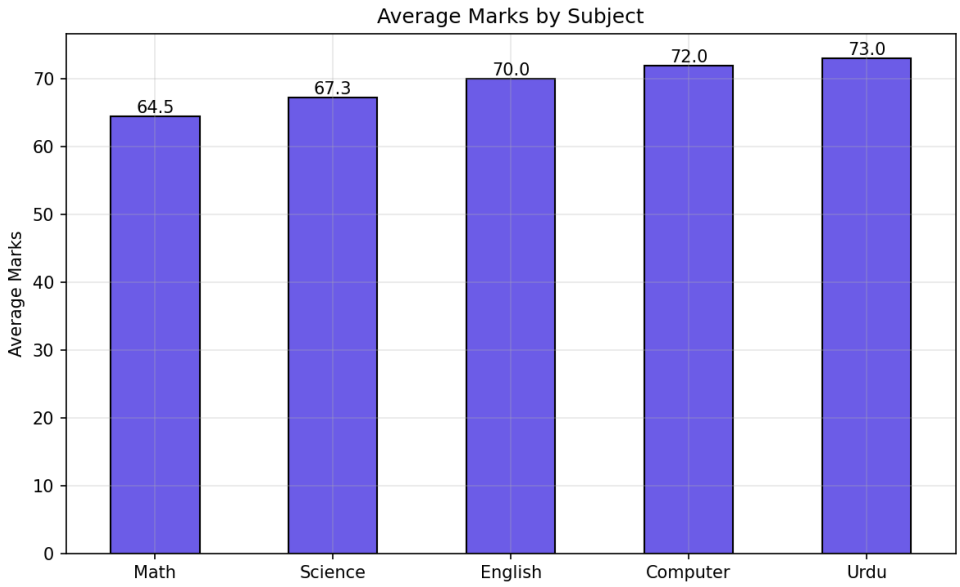
2. Data Preprocessing

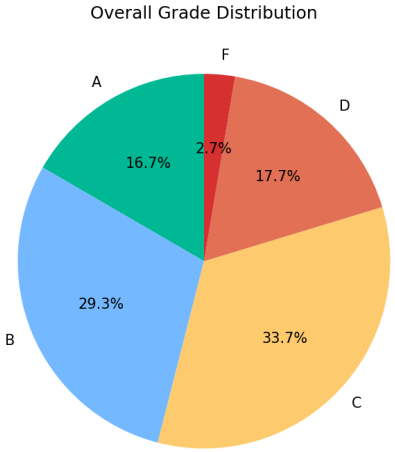
The raw data had 304 rows with 4 duplicate rows and 19 missing values (Math 7, StudyHours 6, Attendance 6). Duplicates were removed and missing values were filled with the column median, leaving 300 clean records. New columns were created: Total, Average, Grade (A-F), Result (Pass/Fail) and Attendance Level.

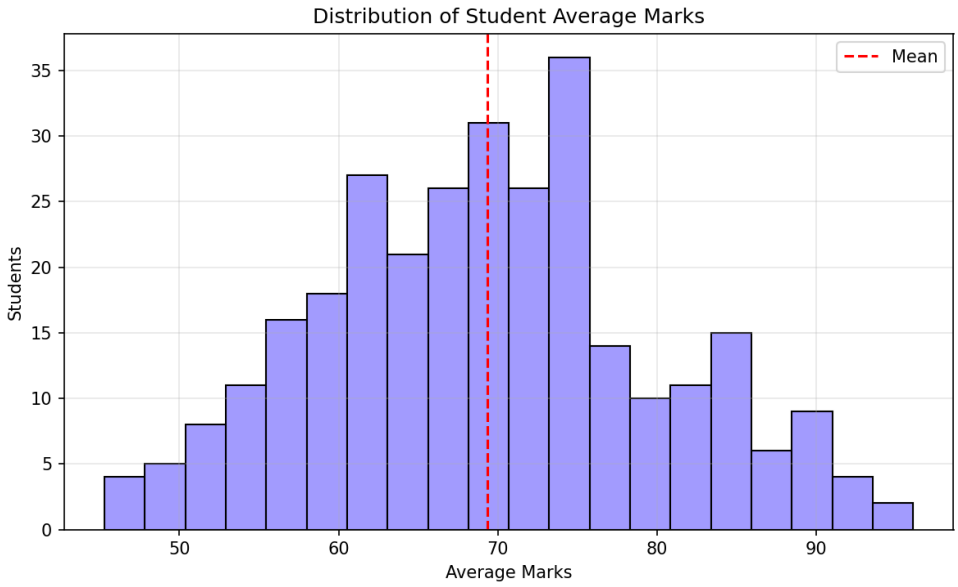
3. Subject-wise and Overall Performance

Subject	Mean	Median	Max	Min
Math	64.5	64.6	99.8	32.8
English	70.0	69.5	100.0	40.9
Science	67.3	66.3	99.9	34.6
Urdu	73.0	73.8	100.0	43.8
Computer	72.0	71.3	100.0	41.8

Overall average: 69.4/100. Pass rate: 97.3%. Urdu is the strongest subject and Math the weakest.

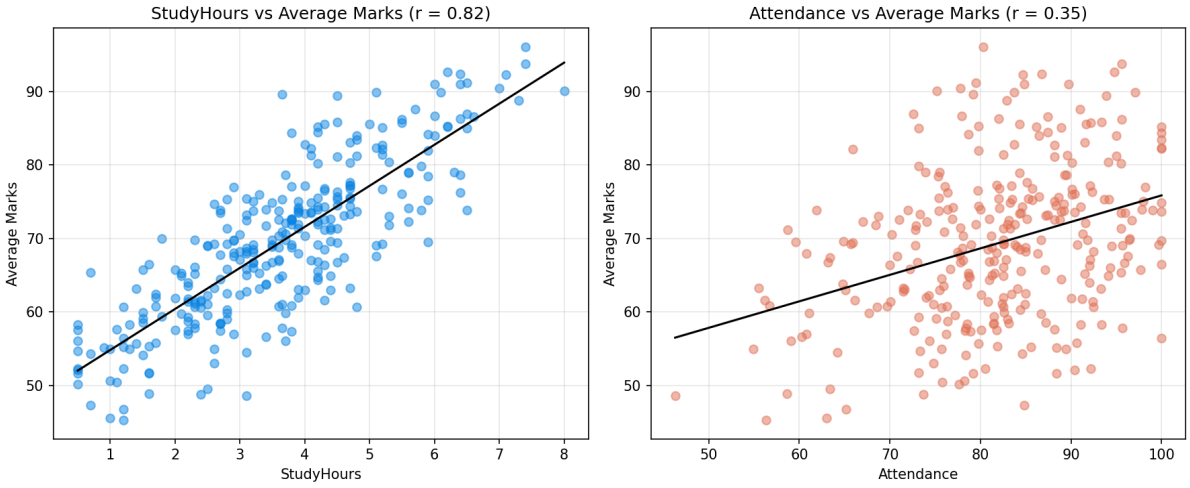


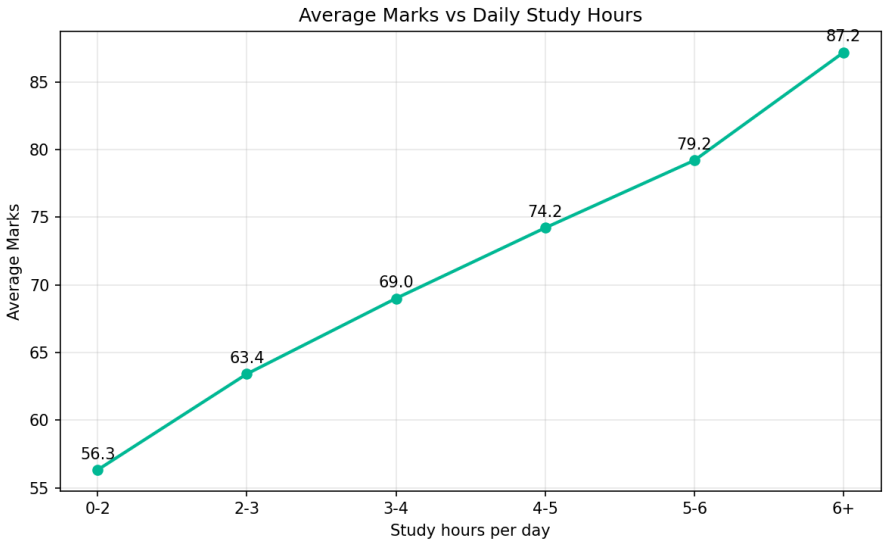

Overall Grade Distribution
F
A
D
2.7%
16.7%
17.7%
29.3%
33.7%
B
C



4. Factors Affecting Performance

Correlation with average marks: Study hours r = 0.82 (strong positive), Attendance r = 0.35 (moderate positive). Students studying 5+ hours/day average 82.5 marks versus 55.9 for those studying under 2 hours. Students with over 90% attendance average 73.0 versus 63.9 for those below 75%.

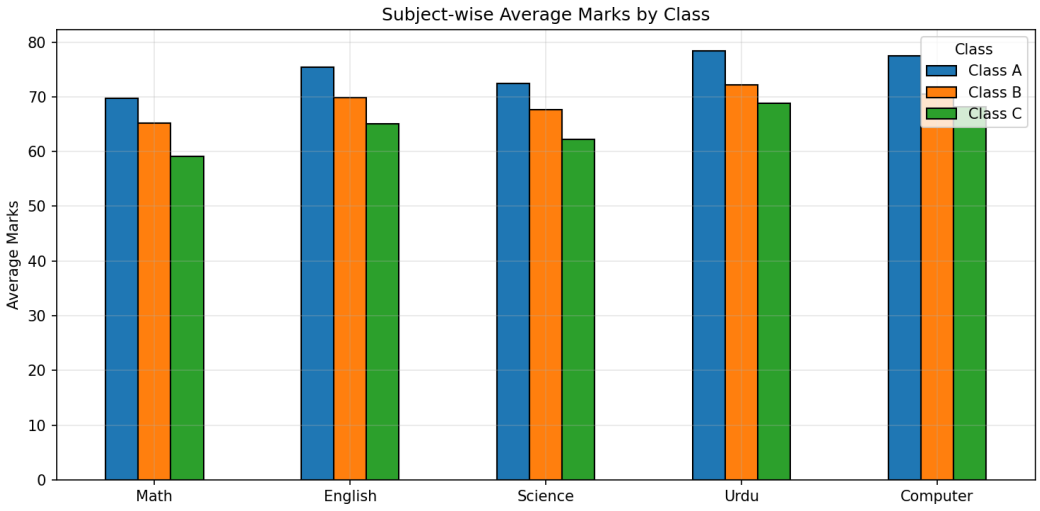




5. Class Comparison

Class	Math	English	Science	Urdu	Computer	Average
Class A	69.7	75.5	72.5	78.3	77.5	74.7
Class B	65.2	69.9	67.6	72.2	70.5	69.1
Class C	59.1	65.1	62.2	68.8	68.2	64.7

Class A performs best (74.7), Class B is average (69.1) and Class C is lowest (64.7) with a 92.5% pass rate, the only class with failures.



6. Insights and Recommendations

- Study hours are the strongest driver of marks; encourage 4-5 hours of structured study daily.
- Low attendance (below 75%) is linked to lower marks; set a minimum attendance policy and follow up early.
- Math is the weakest subject; run remedial and practice sessions.
- Class C needs extra teacher support and peer learning with Class A.
- Identify at-risk students (low study hours and low attendance) early and provide mentoring.

7. Conclusion

Study hours and attendance are the two main factors behind academic results. Targeted support for the weakest subject and lowest-performing class, with attendance monitoring, can improve overall outcomes.

Appendix - Python Code

```
# # Project 2 — Student Performance Analysis
# **Author:** Rizwan Ali  |  **Internship:** Data Analyst, Syntecxhub
#
# **Objectives**
# 1. Load and preprocess the student dataset (marks, attendance, etc.)
# 2. Analyze subject-wise and overall performance trends
# 3. Identify factors affecting performance (study hours, attendance)
# 4. Compare performance across groups/classes
# 5. Visualize with bar, pie and line charts
# 6. Generate insights for improving academic results
#
# > **Note:** The dataset is a synthetic (simulated) student dataset of 300 students generated with a fixed random
    seed, so results are reproducible. To use real data, replace the `students.csv` loading step with your own file
    with the same columns.

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import os

plt.rcParams['figure.figsize'] = (8, 5)
plt.rcParams['axes.grid'] = True
plt.rcParams['grid.alpha'] = 0.3
os.makedirs('charts', exist_ok=True)
SUBJECTS = ['Math', 'English', 'Science', 'Urdu', 'Computer']

# ## 1. Create / Load the Dataset

np.random.seed(42)
n = 300
df = pd.DataFrame({
    'StudentID': [f'S{i:03d}' for i in range(1, n + 1)],
    'Class': np.random.choice(['Class A', 'Class B', 'Class C'], n),
    'Gender': np.random.choice(['Male', 'Female'], n),
    'StudyHours': np.round(np.clip(np.random.normal(3.5, 1.5, n), 0.5, 8), 1),
    'Attendance': np.round(np.clip(np.random.normal(82, 10, n), 45, 100), 1),
})

# Marks depend on study hours + attendance + some random noise
base = 20 + 5.5 * df['StudyHours'] + 0.35 * df['Attendance']
class_bonus = df['Class'].map({'Class A': 4, 'Class B': 0, 'Class C': -3})
subject_shift = {'Math': -5, 'English': 2, 'Science': -2, 'Urdu': 4, 'Computer': 3}
for s in SUBJECTS:
    df[s] = np.round(np.clip(base + class_bonus + subject_shift[s] + np.random.normal(0, 7, n), 0, 100), 1)

# Introduce a few realistic data problems to preprocess
for col in ['Math', 'Attendance', 'StudyHours']:
    df.loc[np.random.choice(n, 6, replace=False), col] = np.nan
df = pd.concat([df, df.sample(4, random_state=1)], ignore_index=True)  # duplicates

df.to_csv('students.csv', index=False)
df = pd.read_csv('students.csv')
df.head()

# ## 2. Preprocessing

print('Shape:', df.shape)
print('\nMissing values:\n', df.isnull().sum()[df.isnull().sum() > 0])
print('\nDuplicate rows:', df.duplicated().sum())

# Remove duplicates
df = df.drop_duplicates().reset_index(drop=True)

# Fill missing numeric values with the median
for col in ['Math', 'Attendance', 'StudyHours']:
    df[col] = df[col].fillna(df[col].median())

# Feature engineering
df['Total'] = df[SUBJECTS].sum(axis=1)
df['Average'] = df[SUBJECTS].mean(axis=1).round(2)

def grade(x):
    if x >= 80: return 'A'
    if x >= 70: return 'B'
    if x >= 60: return 'C'
    if x >= 50: return 'D'
    return 'F'

df['Grade'] = df['Average'].apply(grade)
df['Result'] = np.where(df['Average'] >= 50, 'Pass', 'Fail')
df['AttendanceLevel'] = pd.cut(df['Attendance'], bins=[0, 75, 90, 100],
```

```
                              labels=['Low (<75%)', 'Medium (75-90%)', 'High (>90%)'])

print('Shape after cleaning:', df.shape)
print('Missing values left:', df.isnull().sum().sum())
df.describe().round(2)

# ## 3. Subject-wise and Overall Performance

subject_stats = df[SUBJECTS].agg(['mean', 'median', 'max', 'min', 'std']).T.round(2)
print(subject_stats)
print('\nOverall average marks:', round(df['Average'].mean(), 2))
print('Pass rate: %.1f%%' % ((df['Result'] == 'Pass').mean() * 100))

# BAR CHART — average marks per subject
avg = df[SUBJECTS].mean().sort_values()
ax = avg.plot(kind='bar', color='#6C5CE7', edgecolor='black')
plt.title('Average Marks by Subject'); plt.ylabel('Average Marks'); plt.xticks(rotation=0)
for i, v in enumerate(avg): plt.text(i, v + 0.5, f'{v:.1f}', ha='center')
plt.tight_layout(); plt.savefig('charts/1_subject_avg_bar.png', dpi=150); plt.close()

# PIE CHART — grade distribution
order = ['A', 'B', 'C', 'D', 'F']
gc = df['Grade'].value_counts().reindex(order).fillna(0)
gc = gc[gc > 0]
plt.pie(gc, labels=gc.index, autopct='%1.1f%%', startangle=90,
        colors=['#00B894', '#74B9FF', '#FDCB6E', '#E17055', '#D63031'][:len(gc)])
plt.title('Overall Grade Distribution'); plt.tight_layout()
plt.savefig('charts/2_grade_pie.png', dpi=150); plt.close()

# HISTOGRAM — distribution of average marks
plt.hist(df['Average'], bins=20, color='#A29BFE', edgecolor='black')
plt.axvline(df['Average'].mean(), color='red', linestyle='--', label='Mean')
plt.title('Distribution of Student Average Marks'); plt.xlabel('Average Marks'); plt.ylabel('Students');
    plt.legend()
plt.tight_layout(); plt.savefig('charts/3_average_hist.png', dpi=150); plt.close()

# ## 4. Factors Affecting Performance (Study Hours & Attendance)

corr = df[['StudyHours', 'Attendance', 'Average']].corr().round(3)
print(corr)

# SCATTER + trend lines
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
for ax, col, color in zip(axes, ['StudyHours', 'Attendance'], ['#0984E3', '#E17055']):
    ax.scatter(df[col], df['Average'], alpha=0.5, color=color)
    m, b = np.polyfit(df[col], df['Average'], 1)
    xs = np.linspace(df[col].min(), df[col].max(), 100)
    ax.plot(xs, m * xs + b, color='black')
    ax.set_xlabel(col); ax.set_ylabel('Average Marks')
    ax.set_title(f'{col} vs Average Marks (r = {df[col].corr(df["Average"]):.2f})')
plt.tight_layout(); plt.savefig('charts/4_factors_scatter.png', dpi=150); plt.close()

# LINE CHART — average marks by study hours
df['StudyBand'] = pd.cut(df['StudyHours'], bins=[0, 2, 3, 4, 5, 6, 8],
                         labels=['0-2', '2-3', '3-4', '4-5', '5-6', '6+'])
line = df.groupby('StudyBand', observed=True)['Average'].mean()
plt.plot(line.index.astype(str), line.values, marker='o', color='#00B894', linewidth=2)
for x, y in zip(line.index.astype(str), line.values): plt.text(x, y + 0.8, f'{y:.1f}', ha='center')
plt.title('Average Marks vs Daily Study Hours'); plt.xlabel('Study hours per day'); plt.ylabel('Average Marks')
plt.tight_layout(); plt.savefig('charts/5_studyhours_line.png', dpi=150); plt.close()

att = df.groupby('AttendanceLevel', observed=True)['Average'].mean().round(2)
print('\nAverage marks by attendance level:\n', att)

# ## 5. Compare Performance Across Groups / Classes

class_avg = df.groupby('Class')[SUBJECTS + ['Average']].mean().round(2)
print(class_avg)
print('\nPass rate by class (%):')
print((df.assign(p=df['Result'] == 'Pass').groupby('Class')['p'].mean() * 100).round(1))

# GROUPED BAR CHART — subject averages by class
class_avg[SUBJECTS].T.plot(kind='bar', figsize=(10, 5), edgecolor='black')
plt.title('Subject-wise Average Marks by Class'); plt.ylabel('Average Marks'); plt.xticks(rotation=0)
plt.legend(title='Class'); plt.tight_layout(); plt.savefig('charts/6_class_comparison.png', dpi=150); plt.close()

# Gender comparison + top / bottom students
print(df.groupby('Gender')['Average'].mean().round(2))
print('\nTop 5 students:'); print(df.nlargest(5, 'Average')[['StudentID', 'Class', 'Average', 'Grade']])
print('\nBottom 5 students:'); print(df.nsmallest(5, 'Average')[['StudentID', 'Class', 'StudyHours', 'Attendance',
    'Average']])

# ## 6. Insights and Recommendations
weakest = df[SUBJECTS].mean().idxmin()
```

```
strongest = df[SUBJECTS].mean().idxmax()
best_class = df.groupby('Class')['Average'].mean().idxmax()
worst_class = df.groupby('Class')['Average'].mean().idxmin()
r_study = df['StudyHours'].corr(df['Average'])
r_att = df['Attendance'].corr(df['Average'])
fail_n = (df['Result'] == 'Fail').sum()
low_att_avg = df[df['Attendance'] < 75]['Average'].mean()
high_att_avg = df[df['Attendance'] > 90]['Average'].mean()
low_study_avg = df[df['StudyHours'] < 2]['Average'].mean()
high_study_avg = df[df['StudyHours'] >= 5]['Average'].mean()

insights = f'''
KEY INSIGHTS
1. Overall average marks: {df['Average'].mean():.1f}/100 | Pass rate: {(df['Result']=='Pass').mean()*100:.1f}%
    ({fail_n} students failing).
2. Strongest subject: {strongest} ({df[strongest].mean():.1f}). Weakest subject: {weakest}
    ({df[weakest].mean():.1f}).
3. Study hours have a strong positive correlation with marks (r = {r_study:.2f}). Students studying 5+ hrs/day
    average
   {high_study_avg:.1f} vs {low_study_avg:.1f} for those studying under 2 hrs.
4. Attendance also matters (r = {r_att:.2f}). Students with >90% attendance average {high_att_avg:.1f}
   vs {low_att_avg:.1f} for those below 75%.
5. {best_class} performs best and {worst_class} performs lowest.

RECOMMENDATIONS
- Run extra remedial/practice sessions for {weakest}.
- Set a minimum 75% attendance policy and follow up early with low-attendance students.
- Encourage structured study of at least 4-5 hours/day; provide study guides and timetables.
- Give {worst_class} additional teacher support and peer-learning with {best_class}.
- Identify at-risk students (low study hours + low attendance) early and provide mentoring.
'''
print(insights)
open('insights.txt', 'w').write(insights)
df.to_csv('students_cleaned.csv', index=False)

# ## 7. Conclusion
# Study hours and attendance are the two strongest drivers of academic results in this dataset. Targeted support
    for the weakest subject and the lowest-performing class, along with attendance monitoring, can meaningfully
    improve overall outcomes.
```